LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

STACK DAN QUEUE

Oleh:

Jesica Olivia Putri NIM. 2510817120018

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Jesica Olivia Putri
NIM : 2510817120018

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani NIM. 2310817310009	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN ii

DAFTAR ISI iii

DAFTAR GAMBAR iv

DAFTAR TABEL v

SOAL 1 1

A. Source Code 1
B. Pembahasan 2

SOAL 2 7

A. Source Code 8
B. Output Program 9
C. Pembahasan 9

SOAL 3 12

A. Source Code 12
B. Pembahasan 14

SOAL 4 19

A. Source Code 20
B. Output Program 21
C. Pembahasan 21

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Jawaban Soal 2 9
Gambar 2 Screenshot Hasil Jawaban Soal 4 21

DAFTAR TABEL

Table 1 Source Code stack.h 1
Table 2 Source Code stack.cpp 1
Table 3 Source Code prob1.cpp 8
Table 4 Source Code queue.h 12
Table 5 Source Code queue.cpp 12
Table 6 Source Code prob2.cpp 20

SOAL 1

Implementasi Stack

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 1 Source Code stack.h

```
#ifndef STACK_H
#define STACK_H

#define MAX 100

struct Stack {
    int data[MAX];
    int* top;
};

void init(Stack* s);
bool isEmpty(const Stack* s);
bool isFull(const Stack* s);
void push(Stack* s, int value);
void pop(Stack* s);
int peek(const Stack* s);

#endif
```

Table 2 Source Code stack.cpp

```
#include "stack.h"
#include <stdexcept>

void init(Stack* s){
    s->top = s->data - 1;
}

bool isEmpty(const Stack* s){
    return s->top < s->data;
```

```
10 }
11
12 bool isFull(const Stack* s){
13     return s->top == s->data + MAX - 1;
14 }
15
16 void push(Stack* s, int value){
17     if (isFull(s))
18         throw std::overflow_error("Stack penuh!");
19
20     s->top++;
21     *(s->top) = value;
22 }
23
24 void pop(Stack* s){
25     if (isEmpty(s))
26         throw std::underflow_error("Stack kosong!");
27     s->top--;
28 }
29
30 int peek(const Stack* s){
31     if (isEmpty(s))
32         throw std::underflow_error("Stack kosong!");
33     return *(s->top);
34 }
```

B. Pembahasan

Pada source code ***stack.h***, file ini berfungsi sebagai header yang berisi deklarasi struktur data dan fungsi yang akan digunakan pada file lain. Di dalamnya terdapat konstanta `MAX` yang digunakan sebagai kapasitas maksimal stack yaitu sebanyak 100 elemen. Selain itu, ada `struct Stack` yang memiliki dua jenis, yaitu array `data[MAX]` bertipe `int` untuk menyimpan elemen stack dan pointer `top` bertipe `int*` yang berfungsi untuk menunjuk elemen paling atas dari stack. Header ini juga mendeklarasikan beberapa fungsi yaitu `init, isEmpty, isFull, push, pop,` dan `peek` yang akan diimplementasikan pada file ***stack.cpp***.

Pada source code ***stack.cpp***, baris [1] `#include "stack.h"` digunakan untuk menghubungkan file ini dengan header agar bisa menggunakan struct dan fungsi yang sudah dideklarasikan. Baris [2] `#include <stdexcept>` digunakan untuk menangani kondisi error menggunakan mekanisme exception handling, seperti saat stack

penuh dengan `std::overflow_error` atau saat stack kosong dengan `std::underflow_error`.

- Fungsi `init`

  Pada baris [4], dibuat fungsi `void init(Stack* s)` yang berfungsi untuk menginisialisasi stack agar siap digunakan dalam kondisi kosong. Pada baris [5], `s->top = s->data - 1;` mengatur pointer `top` ke satu posisi sebelum elemen pertama array, yang menandakan bahwa belum ada elemen yang tersimpan dalam stack.

  Alur algoritmanya :

  - Mengatur posisi `top` ke sebelum elemen pertama array.
  - Menandakan bahwa stack dalam kondisi kosong.

  Justifikasi kebenarannya adalah karena `top` tidak menunjuk ke elemen manapun di dalam array, kondisi ini valid sebagai penanda stack kosong yang bisa dicek dengan membandingkan posisi `top` terhadap `data`. Saat dilakukan operasi `push`, elemen pertama akan masuk ke posisi awal array. Kompleksitas waktu fungsi ini adalah O(1) karena hanya melakukan satu operasi assignment, dan kompleksitas ruangnya juga O(1) karena tidak menggunakan memori tambahan.

- Fungsi `isEmpty`

  Pada baris [8], dibuat fungsi `bool isEmpty(const Stack* s)` yang berfungsi untuk mengecek apakah stack dalam keadaan kosong. Pada baris [9], `return s->top < s->data;` digunakan untuk membandingkan apakah posisi `top` masih berada sebelum elemen pertama array, yang menandakan bahwa tidak ada elemen di dalam stack.

  Alur algoritmanya :

  - Membandingkan posisi `top` dengan alamat awal array.
  - Jika `top < data`, maka stack kosong.

  Justifikasi kebenarannya adalah karena pada kondisi awal setelah fungsi `init`, pointer `top` diatur ke `data - 1`, sehingga kondisi `top < data` pasti terpenuhi dan menandakan stack kosong. Kondisi ini juga berlaku ketika semua elemen sudah

di-pop, sehingga `top` kembali berada sebelum array. Kompleksitas waktu fungsi ini adalah O(1) karena hanya satu operasi perbandingan, dan kompleksitas ruangnya juga O(1) karena tidak menggunakan memori tambahan.

- Fungsi `isFull`

  Pada baris [12], dibuat fungsi `bool isFull(const Stack* s)` yang berfungsi untuk mengecek apakah stack sudah penuh. Pada baris [13], `return s->top == s->data + MAX - 1;` digunakan untuk mengecek apakah posisi `top` sudah berada pada elemen terakhir array, yang menandakan bahwa tidak ada lagi slot kosong di dalam stack.

  Alur algoritmanya :

  • Membandingkan posisi `top` dengan alamat elemen terakhir array.

  • Jika `top == data + MAX - 1`, maka stack penuh.

  Justifikasi kebenarannya adalah karena `s->data + MAX - 1` merupakan alamat elemen terakhir dari array berukuran `MAX`. Ketika `top` sudah menunjuk ke posisi tersebut, maka seluruh kapasitas stack telah terisi dan tidak dapat dilakukan penambahan elemen lagi. Kompleksitas waktu fungsi ini adalah O(1) karena hanya melakukan satu operasi perbandingan, dan kompleksitas ruangnya juga O(1) karena tidak menggunakan memori tambahan.

- Fungsi `push`

  Pada baris [16], dibuat fungsi `void push(Stack* s, int value)` yang berfungsi untuk menambahkan elemen baru ke dalam stack. Pada baris [17-18], dilakukan pengecekan `if (isFull(s))`, jika stack sudah penuh maka akan dilempar exception `overflow std::overflow_error` dengan pesan "Stack penuh!" sebagai penanganan kondisi tidak valid. Pada baris [20], `s->top++;` digunakan untuk menggeser posisi pointer `top` satu langkah ke depan, lalu pada baris [21], `*(s->top) = value;` digunakan untuk menyimpan nilai ke alamat yang ditunjuk oleh `top`.

  Alur algoritmanya :

  • Mengecek apakah stack penuh.

- Jika tidak penuh, menaikkan posisi `top`.
- Menyimpan nilai pada posisi `top`.

Justifikasi kebenarannya adalah karena penambahan elemen pada stack mengikuti prinsip LIFO (Last In First Out), sehingga elemen baru harus ditempatkan di bagian paling atas. Dengan menggeser `top` terlebih dahulu lalu mengisi nilainya, urutan elemen tetap terjaga dengan benar. Selain itu, pengecekan kondisi penuh dilakukan untuk mencegah overflow. Kompleksitas waktu fungsi ini adalah O(1) karena hanya melibatkan operasi sederhana, dan kompleksitas ruangnya juga O(1) karena tidak menggunakan memori tambahan.

- Fungsi `pop`

Pada baris [24], dibuat fungsi `void pop(Stack* s)` yang berfungsi untuk menghapus elemen paling atas dari stack. Pada baris [25-26], dilakukan pengecekan `if (isEmpty(s))`, jika stack dalam kondisi kosong maka akan dilempar exception `underflow_error` dengan pesan "Stack kosong!" sebagai penanganan kondisi tidak valid. Pada baris [27], `s->top--;` digunakan untuk menggeser pointer top satu posisi ke belakang sehingga elemen teratas dianggap sudah tidak ada.

Alur algoritmanya :

- Mengecek apakah stack kosong.
- Jika tidak kosong, menurunkan posisi `top`.

Justifikasi kebenarannya adalah karena dengan menurunkan `top`, elemen terakhir dianggap terhapus karena tidak lagi dapat diakses oleh stack. Meskipun data masih ada di memori, elemen tersebut sudah keluar dari stack. Hal ini sesuai dengan prinsip LIFO (Last In First Out), dimana elemen yang terakhir masuk akan menjadi yang pertama keluar. Kompleksitas waktu fungsi ini adalah O(1) karena hanya melakukan operasi sederhana, dan kompleksitas ruangnya juga O(1) karena tidak menggunakan memori tambahan.

- Fungsi `peek`

Pada baris [30], dibuat fungsi `int peek(const Stack* s)` yang berfungsi untuk melihat nilai elemen paling atas tanpa menghapusnya. Pada baris

[31-32], dilakukan pengecekan `if (isEmpty(s))`, jika stack kosong maka akan dilempar exception `underflow_error`. Pada baris [33], `return *(s->top);` digunakan untuk mengembalikan nilai yang ditunjuk oleh pointer `top` melalui proses dereferencing.

Alur algoritmanya :

- Mengecek apakah stack kosong.
- Jika tidak kosong, mengambil nilai pada posisi `top`.

Justifikasi kebenarannya adalah karena top selalu menunjuk ke elemen terakhir yang di-push ke dalam stack, sehingga nilai yang dikembalikan pasti merupakan elemen paling atas. Fungsi ini tidak mengubah isi stack karena hanya membaca nilai tanpa menghapusnya. Kompleksitas waktu fungsi ini adalah O(1) karena hanya melakukan operasi sederhana, dan kompleksitas ruangnya juga O(1) karena tidak menggunakan memori tambahan.

SOAL 2

Time Limit: 1.0 s
Memory Limit: 64

Si Palui dan Mesin Hitung Aneh

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- A_i adalah nilai bilangan pada input
- Simbol yang valid hanya:
  - bilangan bulat
  - operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Table 3 Source Code prob1.cpp

```
#include "stack.h"
#include <iostream>
#include <sstream>

int main(){
    int n;
    std::cin >> n;

    Stack s;
    init(&s);

    for (int i = 0; i < n; i++){
        std::string token;
        std::cin >> token;

        if (token == "+" || token == "-" || token == "*"
|| token == "/"){
            int b = peek(&s); pop(&s);
            int a = peek(&s); pop(&s);

            int hasil;

```

```
23                if      (token == "+") hasil = a + b;
24                else if (token == "-") hasil = a - b;
25                else if (token == "*") hasil = a * b;
26                else if (token == "/") hasil = a / b;
27
28                push(&s, hasil);
28            } else
30                push(&s, std::stoi(token));
31        }
32
33        std::cout << peek(&s) << std::endl;
34        return 0;
35    }
```

B. Output Program

```
PS C:\Users\ASUS\Documents\GitHub\module-2-stack-and-queue-JesicaOlip> g++ prob1.cpp stack.cpp -o prob1
PS C:\Users\ASUS\Documents\GitHub\module-2-stack-and-queue-JesicaOlip> ./prob1
3
2 3 +
5
PS C:\Users\ASUS\Documents\GitHub\module-2-stack-and-queue-JesicaOlip> ./prob1
9
5 1 2 + 4 * + 3 -
14
PS C:\Users\ASUS\Documents\GitHub\module-2-stack-and-queue-JesicaOlip> ./prob1
5
10 2 / 3 *
15
```

Gambar 1 Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Pada soal ini, mesin hitung bekerja dengan cara membaca ekspresi dalam bentuk operasi yang dituliskan setelah operand. Untuk menyelesaikan perhitungan digunakan struktur data stack karena sesuai dengan prinsip LIFO *(Last In First Out)*, dimana elemen terakhir yang masuk akan menjadi yang pertama diproses.

Pada source code *prob1.cpp*, baris [1] `#include "stack.h"` digunakan untuk menghubungkan file dengan header stack agar dapat menggunakan struktur `Stack` beserta fungsi-fungsinya. Baris [2] `#include <iostream>` digunakan untuk proses input dan output seperti `cin` dan `cout`. Baris [3] `#include <sstream>` disiapkan untuk pengolahan string, meskipun pada implementasi ini konversi dilakukan menggunakan `std::stoi`.

Pada baris [5], program utama dimulai dengan `int main()`. Baris [6], dideklarasikan variabel `int n` untuk menyimpan jumlah simbol yang akan diproses, dan pada baris [7], `std::cin >> n` digunakan untuk menerima input tersebut.

Pada baris [9-10], dibuat variabel `Stack s` dan dilakukan inisialisasi dengan `init(&s)` agar stack berada dalam kondisi kosong sebelum digunakan.

Baris [12], dilakukan perulangan `for (int i = 0; i < n; i++)` untuk membaca dan memproses setiap simbol sebanyak n kali. Pada setiap iterasi, program akan mengambil satu simbol baik berupa bilangan maupun operator, kemudian menentukan apakah simbol tersebut perlu dimasukkan ke dalam stack atau digunakan untuk melakukan operasi terhadap elemen yang sudah ada di dalam stack.

Baris [13-14], `std::string token` dan `std::cin >> token` digunakan untuk membaca setiap simbol input, baik berupa angka maupun operator.

Pada baris [16], dilakukan pengecekan apakah `token` merupakan operator (`+`, `-`, `*`, `/`). Jika token adalah operator, maka pada baris [17], diambil elemen teratas stack sebagai operand kedua dengan `int b = peek(&s);` lalu dihapus menggunakan `pop(&s)`.

Kemudian pada baris [18], diambil elemen berikutnya sebagai operand pertama dengan `int a = peek(&s);` lalu juga dihapus. Urutan penting karena sifat stack adalah LIFO, sehingga operand kedua harus diambil lebih dulu agar hasil operasi tetap benar.

Pada baris [20-24], dibuat variabel `hasil` yang digunakan untuk menyimpan hasil operasi aritmatika sesuai operator yang dibaca. Jika operator + maka dilakukan `a + b`, jika - maka `a - b`, jika * maka `a * b`, dan jika / maka `a / b`. Setelah itu, pada baris [26], hasil dimasukkan kembali ke dalam stack menggunakan `push(&s, hasil)`.

Jika token bukan operator, maka pada baris [27], token dianggap sebagai bilangan, dikonversi ke tipe integer menggunakan `std::stoi(token)`, lalu dimasukkan ke dalam stack menggunakan `push`.

Setelah seluruh simbol selesai diproses, pada baris [30], `std::cout << peek(&s) << std::endl` digunakan untuk menampilkan hasil akhir yang berada

di puncak stack. Baris [31], `return 0` menandakan bahwa program selesai dijalankan dengan sukses.

Kompleksitas waktunya adalah O(n) karena setiap simbol diproses satu kali dalam perulangan. Setiap operasi yang dilakukan di dalam perulangan seperti `push`, `pop`, dan `peek` memiliki kompleksitas waktu O(1), sehingga secara keseluruhan tetap linear terhadap jumlah simbol. Kompleksitas ruangnya adalah O(n) karena dalam kondisi terburuk seluruh simbol berupa bilangan sehingga semuanya akan disimpan di dalam stack sebelum diproses oleh operator.

SOAL 3

Implementasi Queue

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Table 4 Source Code queue.h

```
#ifndef QUEUE_H
#define QUEUE_H

#define MAX 100

struct Queue {
    int data[MAX];
    int* front;
    int* rear;
};

void init(Queue* q);
bool isEmpty(const Queue* q);
bool isFull(const Queue* q);
void enqueue(Queue* q, int value);
void dequeue(Queue* q);
int front(const Queue* q);
int back(const Queue* q);

#endif
```

Table 5 Source Code queue.cpp

```
#include "queue.h"
#include <stdexcept>

void init(Queue* q){
    q->front = q->data;
    q->rear = q->data - 1;
}

```

```
bool isEmpty(const Queue* q){
    return q->front == q->data && q->rear == q->data - 1;
}

bool isFull(const Queue* q){
    if (isEmpty(q)) return false;

    const int* nextRear;

    if (q->rear == q->data + MAX - 1)
        nextRear = q->data;
    else
        nextRear = q->rear + 1;
    return nextRear == q->front;
}

void enqueue(Queue* q, int value){
    if (isFull(q))
        throw std::overflow_error("Queue penuh!");

    if (isEmpty(q)){
        q->front = q->data;
        q->rear = q->data;
    } else {
        if (q->rear == q->data + MAX - 1)
            q->rear = q->data;
        else
            q->rear++;
    }
    *(q->rear) = value;
}

void dequeue(Queue* q){
    if (isEmpty(q))
        throw std::underflow_error("Queue kosong!");

    if (q->front == q->rear){
        q->front = q->data;
        q->rear = q->data - 1;
    } else {
        if (q->front == q->data + MAX - 1)
            q->front = q->data;
        else
            q->front++;
    }
}

```

```
56 int front(const Queue* q){
57     if (isEmpty(q))
58         throw std::underflow_error("Queue kosong!");
59
60     return *(q->front);
61 }
62
63 int back(const Queue* q){
64     if (isEmpty(q))
65         throw std::underflow_error("Queue kosong!");
66
67     return *(q->rear);
68 }
```

B. Pembahasan

Pada source code ***queue.h***, file ini berfungsi sebagai header yang berisi deklarasi struktur data dan fungsi yang akan digunakan pada file lain. Di dalamnya terdapat konstanta `MAX` yang digunakan sebagai kapasitas maksimal queue yaitu sebanyak 100 elemen. Terdapat `struct Queue` yang memiliki tiga komponen, yaitu array data[MAX] bertipe `int` untuk menyimpan elemen queue, serta dua pointer `front` dan `rear` bertipe `int*` yang masing-masing berfungsi untuk menunjuk elemen paling depan dan paling belakang dalam queue. Header ini juga mendeklarasikan fungsi `init`, `isEmpty`, `isFull`, `enqueue`, `dequeue`, `front`, dan `back` yang akan diimplementasikan pada file ***queue.cpp***.

Pada source code ***queue.cpp***, baris [1] `#include "queue.h"` digunakan untuk menghubungkan file dengan header agar dapat menggunakan struct dan fungsi yang telah dideklarasikan sebelumnya. Baris [2] `#include <stdexcept>` digunakan untuk menangani kondisi error menggunakan exception seperti queue penuh (`std::overflow_error`) dan queue kosong (`std::underflow_error`).

- Fungsi `init`

  Pada baris [4], dibuat fungsi `void init(Queue* q)` yang berfungsi untuk menginisialisasi queue dalam kondisi kosong sebelum digunakan.

  Pada baris [5], `q->front = q->data` mengatur pointer `front` agar menunjuk ke alamat elemen pertama array.

Pada baris [6], `q->rear = q->data - 1` mengatur pointer `rear` agar berada satu posisi sebelum elemen pertama array, yang menandakan bahwa belum ada elemen di dalam queue.

Alur algoritmanya adalah mengatur posisi `front` di awal array dan `rear` sebelum elemen pertama array sebagai kondisi awal queue kosong.

Justifikasi kebenarannya adalah karena kombinasi posisi tersebut menjadi penanda kondisi kosong yang konsisten, sehingga dapat dikenali dengan mudah oleh fungsi lain seperti `isEmpty`. Ketika elemen pertama dimasukkan, `rear` akan maju ke posisi awal array dan queue mulai terisi dengan benar. Kompleksitas waktu fungsi ini adalah O(1) dan kompleksitas ruangnya juga O(1).

- Fungsi `isEmpty`

  Pada baris [9], fungsi `bool isEmpty(const Queue* q)` yang berfungsi untuk mengecek apakah queue dalam keadaan kosong.

  Pada baris [10], `return q->front == q->data && q->rear == q->data - 1` digunakan untuk mengecek kondisi kosong, yaitu ketika pointer `front` berada di awal array dan `rear` berada satu posisi sebelum elemen pertama array.

  Alur algoritmanya adalah membandingkan posisi `front` dan `rear` dengan kondisi awal queue setelah inisialisasi.

  Justifikasi kebenarannya adalah karena kondisi ini hanya terpenuhi saat `init` baru dipanggil atau setelah semua elemen telah di-dequeue, sehingga pointer kembali ke posisi awal. Dengan demikian, kondisi tersebut menjadi penanda yang konsisten untuk mendeteksi queue kosong. Kompleksitas waktu fungsi ini adalah O(1) dan kompleksitas ruangnya juga O(1).

- Fungsi `isFull`

  Pada baris [13], fungsi `bool isFull(const Queue* q)` yang berfungsi untuk mengecek apakah queue sudah penuh.

  Pada baris [14], dilakukan pengecekan `if (isEmpty(q)) return false` untuk memastikan bahwa queue yang kosong tidak dianggap penuh. Selanjutnya, pada baris [16-19], dihitung posisi `nextRear`, yaitu posisi berikutnya

dari `rear`. Jika `rear` berada di akhir array, maka `nextRear` akan kembali ke awal array (*wrap-around*), jika tidak maka nilainya adalah `rear + 1`. Pada baris [22], `return nextRear == q->front` digunakan untuk menentukan kondisi penuh.

Alur algoritmanya adalah menentukan posisi berikutnya dari `rear`, kemudian membandingkannya dengan posisi `front`.

Justifikasi kebenarannya adalah karena queue menggunakan konsep circular, sehingga ketika posisi berikutnya dari `rear` sudah sama dengan `front`, berarti seluruh kapasitas telah terisi dan tidak ada lagi ruang untuk menambahkan elemen baru. Pengecekan *wrap-around* memastikan perhitungan posisi tetap benar meskipun `rear` berada di ujung array. Kompleksitas waktu fungsi ini adalah O(1), dan kompleksitas ruangnya juga O(1).

- Fungsi `enqueue`

  Pada baris [25], fungsi `void enqueue(Queue* q, int value)` yang berfungsi untuk menambahkan elemen baru ke dalam queue.

  Pada baris [26-27], dilakukan pengecekan `if (isFull(q))`, jika queue sudah penuh maka akan dilempar exception `overflow_error` dengan pesan "Queue penuh!" sebagai penanganan kondisi tidak valid.

  Pada baris [29-31], jika queue dalam kondisi kosong, maka `front` dan `rear` diatur ke awal array karena elemen pertama yang dimasukkan akan menjadi sekaligus elemen terdepan dan terbelakang.

  Selanjutnya pada baris [33-36], jika queue tidak kosong, maka rear digeser ke posisi berikutnya, yaitu jika `rear` berada di akhir array maka akan kembali ke awal array (*wrap-around*), jika tidak maka cukup dilakukan `rear++`.

  Pada baris [35], `*(q->rear) = value` digunakan untuk menyimpan nilai ke alamat yang ditunjuk oleh rear.

  Alur algoritmanya adalah mengecek apakah queue penuh, kemudian menentukan posisi baru rear, dan terakhir menyimpan nilai pada posisi tersebut.

  Justifikasi kebenarannya adalah karena penambahan elemen pada queue mengikuti prinsip FIFO (First In First Out), sehingga elemen baru selalu dimasukkan

di bagian belakang (`rear`). Dengan menangani pergeseran `rear` dan kondisi *wrap-around*, array dapat digunakan secara melingkar sehingga slot yang sudah kosong akibat operasi dequeue dapat digunakan kembali. Kompleksitas waktu fungsi ini adalah O(1) karena hanya melibatkan operasi sederhana, dan kompleksitas ruangnya juga O(1) karena tidak menggunakan memori tambahan.

- Fungsi `dequeue`

  Pada baris [41], `void dequeue(Queue* q)` yang berfungsi untuk menghapus elemen dari bagian depan queue.

  Pada baris [42-43], dilakukan pengecekan `if (isEmpty(q))`, jika queue dalam kondisi kosong maka akan dilempar exception `underflow_error` dengan pesan "Queue kosong!" sebagai penanganan kondisi tidak valid.

  Pada baris [45-47], jika `front` dan `rear` menunjuk ke alamat yang sama, artinya hanya terdapat satu elemen di dalam queue. Setelah elemen tersebut dihapus, maka queue dikembalikan ke kondisi awal, yaitu `front` ke awal array dan `rear` ke satu posisi sebelum elemen pertama array.

  Baris [49-53], jika terdapat lebih dari satu elemen, maka `front` digeser ke posisi berikutnya dengan memperhatikan konsep circular, yaitu jika `front` berada di ujung array maka akan kembali ke awal. jika tidak maka dilakukan `front++`.

  Alur algoritmanya adalah mengecek kondisi kosong, kemudian menentukan apakah hanya terdapat satu elemen atau lebih, lalu menggeser posisi `front` sesuai kondisi.

  Justifikasi kebenarannya adalah karena penghapusan elemen pada queue mengikuti prinsip FIFO (First In First Out), sehingga elemen yang pertama masuk akan menjadi yang pertama keluar. Dengan menggeser front ke depan, elemen terdepan dianggap sudah keluar dari queue. Kompleksitas waktu adalah O(1) dan kompleksitas ruangnya juga O(1).

- Fungsi `front`

  Pada baris [56], `int front(const Queue* q)` berfungsi untuk mengambil nilai elemen paling depan tanpa menghapusnya.

Pada baris [57-58], dilakukan pengecekan `if (isEmpty(q))`, jika queue dalam kondisi kosong maka akan dilempar exception `underflow_error` sebagai penanganan kondisi tidak valid.

Pada [54], `return *(q->front)` untuk mengembalikan nilai yang ditunjuk oleh pointer `front`.

Alur algoritmanya adalah mengecek kondisi kosong, kemudian mengambil nilai pada posisi `front`.

Justifikasi kebenarannya adalah karena pointer `front` selalu menunjuk ke elemen yang paling awal masuk ke dalam queue, sehingga nilai yang dikembalikan merupakan elemen terdepan yang benar tanpa mengubah isi queue. Fungsi ini hanya membaca nilai tanpa menghapusnya, sehingga tidak mempengaruhi struktur data. Kompleksitas waktu fungsi ini adalah O(1) dan kompleksitas ruangnya juga O(1).

- Fungsi `back`

  Baris [63] `int back(const Queue* q)` yang berfungsi untuk mengambil nilai elemen paling belakang tanpa menghapusnya.

  Pada baris [64-65], pengecekan `if (isEmpty(q))`, jika queue dalam kondisi kosong maka akan dilempar exception `underflow_error` sebagai penanganan kondisi tidak valid.

  [61], `return *(q->rear)` digunakan untuk mengembalikan nilai yang ditunjuk oleh pointer rear melalui proses dereferencing.

  Justifikasi kebenarannya adalah karena pointer `rear` selalu menunjuk ke elemen yang paling akhir dimasukkan ke dalam queue, sehingga nilai yang dikembalikan merupakan elemen terbelakang yang benar tanpa mengubah isi queue. Kompleksitas waktu fungsi ini adalah O(1) dan kompleksitas ruangnya juga O(1).

SOAL 4

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui dan Deretan Angka

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq Ai \leq 1000$

Keterangan:

- n: jumlah elemen
- k: ukuran jendela
- A_i: elemen array

Format Input

n k

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok ayng dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Table 6 Source Code prob2.cpp

```
#include "queue.h"
#include <iostream>

int main(){
    int n, k;
    std::cin >> n >> k;

    int arr[100];
    for (int i = 0; i < n; i++)
        std::cin >> arr[i];

    Queue q;
    init(&q);

    int sum = 0;
    for (int i = 0; i < k; i++) {
        enqueue(&q, arr[i]);
        sum += arr[i];
    }

    std::cout << sum;

    for (int i = k; i < n; i++) {
        sum -= front(&q);
        dequeue(&q);

        enqueue(&q, arr[i]);
        sum += arr[i];

        std::cout << " " << sum;
    }

```

```
    std::cout << std::endl;
    return 0;
}
```

B. Output Program

```
PS C:\Users\ASUS\Documents\GitHub\module-2-stack-and-queue-JesicaOlip> g++ prob2.cpp queue.cpp -o prob2
PS C:\Users\ASUS\Documents\GitHub\module-2-stack-and-queue-JesicaOlip> ./prob2
5 3
1 2 3 4 5
6 9 12
PS C:\Users\ASUS\Documents\GitHub\module-2-stack-and-queue-JesicaOlip> ./prob2
5 5
1 2 3 4 5
15
PS C:\Users\ASUS\Documents\GitHub\module-2-stack-and-queue-JesicaOlip> ./prob2
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
```

Gambar 2 Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Pada soal ini, dilakukan perhitungan jumlah dari beberapa elemen yang berurutan sebanyak k, kemudian posisi perhitungan tersebut digeser satu langkah ke kanan dan dihitung kembali hingga tidak memungkinkan lagi. Untuk menyelesaikan perhitungan ini digunakan struktur data queue karena sesuai dengan prinsip FIFO (First In First Out), di mana elemen yang pertama masuk akan menjadi yang pertama keluar, sehingga cocok untuk merepresentasikan pergeseran window.

Pada source code ***prob2.cpp***, baris [1] `#include "queue.h"` digunakan untuk menghubungkan file ini dengan header queue agar dapat menggunakan struktur Queue beserta fungsi-fungsinya.

Baris [2], `#include <iostream>` digunakan untuk mendukung operasi input dan output seperti cin dan cout.

Baris [4], dimulai dengan `int main()`. Baris [5], dideklarasikan dua variabel `int n` dan `int k`, dimana `n` menyimpan jumlah elemen array dan `k` menyimpan ukuran jendela.

Pada baris [6], `std::cin >> n >> k` digunakan untuk menerima kedua input tersebut sekaligus.

Pada baris [8], dideklarasikan array `int arr[100]` untuk menyimpan seluruh elemen.

Selanjutnya pada baris [9-10], dilakukan perulangan `for` untuk membaca semua elemen input ke dalam array.

Pada baris [12-13], dideklarasikan variabel `Queue q` dan dilakukan inisialisasi dengan `init(&q)` agar queue berada dalam kondisi kosong sebelum digunakan.

Pada baris [15], dideklarasikan variabel `int sum = 0` untuk menyimpan jumlah elemen saat ini.

Pada baris [16-18], dilakukan perulangan pertama untuk mengisi jendela awal sebanyak k elemen. Setiap elemen dimasukkan ke dalam queue menggunakan `enqueue` dan nilainya ditambahkan ke `sum`.

Setelah perulangan selesai, pada baris [21], `std::cout << sum` digunakan untuk mencetak jumlah dari jendela pertama.

Pada baris [23-30], dilakukan perulangan kedua untuk menggeser jendela ke kanan sebanyak n - k kali.

Pada baris [24], `sum -= front(&q)` digunakan untuk mengurangi nilai sum dengan elemen terdepan yang akan keluar dari jendela.

Baris [21], `dequeue(&q)` menghapus elemen tersebut dari queue. Pada baris [27], `enqueue(&q, arr[i])` menambahkan elemen baru yang masuk ke jendela.

Pada baris [28] `sum += arr[i]` menambahkan nilai elemen baru tersebut ke dalam sum. Pada baris [30] `std::cout << " " << sum` mencetak hasil jumlah jendela saat ini dengan dipisahkan oleh spasi.

Pada baris [33], `std::cout << std::endl` digunakan untuk mencetak baris baru setelah seluruh hasil ditampilkan. Baris [35], `return 0;` menandakan bahwa program selesai dijalankan dengan sukses.

Kompleksitas waktunya adalah O(n) karena perulangan pertama berjalan sebanyak k kali dan perulangan kedua berjalan sebanyak n - k kali, sehingga totalnya tetap n iterasi. Setiap operasi di dalam perulangan seperti `enqueue`, `dequeue`, dan `front` memiliki kompleksitas waktu O(1), sehingga secara keseluruhan tetap linear terhadap jumlah elemen. Kompleksitas ruangnya adalah O(k) karena `queue` hanya menyimpan elemen sebanyak ukuran jendela, yaitu k elemen pada setiap saat, tidak bergantung pada besar nilai n.